1. The complete ClaimSathi AI product flow
ClaimSathi AI
Customer dashboard → AI claims assistant → Evidence screening → Professional report
1. Customer Registration / Login
Unique user ID · Account · Previous claims

2. AI Claims Conversation
Voice and text · Gnani STT/TTS · Multilingual interaction

3. Claim Information Extraction
Incident details · Claim category · Missing-field detection · Follow-up questions

4. Evidence Collection
Image uploads · Video uploads · Guided live camera capture

5. Basic Evidence Screening
AI provenance signals · Duplicate detection · Image integrity checks

6. Customer Review & Confirmation
Correct details · Review evidence findings · Confirm report information

7. Professional Claim Report Generator
Structured data · Evidence inventory · Incident summary · PDF report

8. Customer Claims Vault
Unique claim ID · Download report · Evidence records · Status history

Future phase — insurer partnership
9. Insurer Integration
Secure submission · Insurer acknowledgement · Official claim number · Status tracking

This should be our single source of truth for the product workflow.
2. The most important technical component: Evidence screening
You explicitly don't want an elaborate deepfake investigation system in the MVP. Agreed.

Let's build a focused evidence-integrity module with four checks.
A. AI-generated image screening
We should start with content provenance verification.

Check for:
- C2PA Content Credentials.
- Supported SynthID watermarks.
- Available image metadata.
- Signs of invalid or tampered provenance credentials.

There are useful existing technologies:
- C2PA — signed content-origin and editing-history credentials.
- OpenAI Verify — checks supported provenance signals.
- Google SynthID — watermarking and identification for supported AI-generated content.
An important technical limitation: OpenAI's current Content Provenance API checks supported C2PA and SynthID signals; it is not a universal AI-image detector. A not_detected result does not establish that an image is authentic.
OpenAI API+1

Google also announced broader public access to SynthID Detector on October 29, 2026, but its detection is still based on supported provenance signals.
blog.google

MVP implementation rule:
Result System response
Supported AI watermark detected AI provenance detected
Valid AI-generation credential detected AI-generated provenance detected
Invalid credential Integrity warning
No supported signal Inconclusive
Metadata missing Metadata unavailable

Do not classify every image without a watermark as genuine.
B. Duplicate image detection
High-priority MVP feature
This is something we can implement directly.

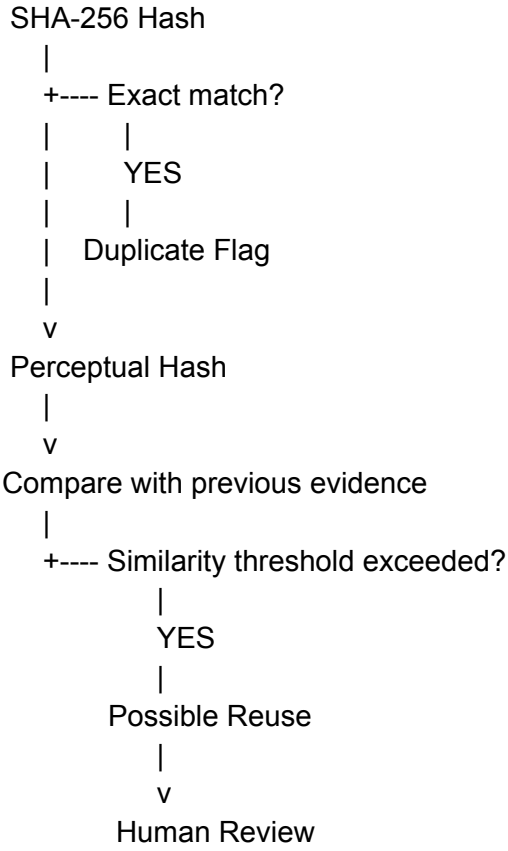
Suppose Rahul uploads a damaged-bike photograph for claim 001.

Three weeks later, the same image is uploaded for claim 019.

We need to identify this.

Use two algorithms:
Algorithm Purpose
SHA-256 Detect exactly identical files
Perceptual Hash (pHash) Detect visually similar images, including resized or recompressed copies

Example:
Evidence Upload
|
v



Important implementation details:
- Compare evidence across previous claims, not just within one claim.
- Identify whether a match belongs to the same claim or a different claim.
- Store the matching claim reference and comparison result.
- Never expose another customer's private claim information to the current customer.
- Use an appropriately protected cross-claim fingerprint index.

C. Basic file integrity checks

Every uploaded image or video should pass:
- File signature and MIME-type verification.
- File size and format restrictions.
- SHA-256 hashing.
- Metadata extraction, where available.
- Upload timestamp recording.
- Malware scanning.
- Original file preservation.

Do not rely on the filename or browser-provided content type to establish the actual file format.

D. Live camera evidence

This is a valuable feature for the MVP.

The customer can choose:
1. Upload existing photographs.
2. Upload an existing video.
3. Capture new photographs through the ClaimSathi dashboard.
4. Record a guided video through the dashboard.
For live capture, implement:
- Camera permission.
- A server-generated capture session ID.
- Randomized capture instructions.
- Server-side receipt timestamp.
- Original file hashing.
- Capture-session association with the claim.

For example, the system asks the customer to record a short video showing the front, left side and right side of the damaged vehicle.

This creates better evidence provenance than a normal upload, but it does not prove the photographed scene is genuine. Someone could still photograph a displayed AI image or stage a scene.

Video limitation for MVP: We can validate video files, identify exact duplicate videos and inspect basic metadata. Full frame-by-frame synthetic-video detection is not required.
3. What exactly does our AI generate?
This should become the central deliverable of ClaimSathi AI.

I recommend generating a Professional Insurance Claim Dossier.

Not merely a conversational summary. A structured case package.
CLAIM DOSSIER
CLAIMSATHI AI · SAMPLE OUTPUT

Customer confirmed

CS-2026-000124
Motor Insurance · Report version 1.0

SECTION 01 — Customer Information

Name, unique customer ID, contact information and declared insurance provider.

SECTION 02 — Policy Information

Policy number, insurer name, policy type and available policy details. Unverified customer-entered values are clearly marked.

SECTION 03 — Incident Information

Incident date and time, location, incident description, vehicle details, people involved and customer-reported injuries or damage.

SECTION 04 — Detailed Incident Statement

A professionally written chronological account generated from the customer's conversation and confirmed by the customer.

SECTION 05 — Evidence Inventory
Evidence Screening outcome

Image 01	No supported AI signal found
Image 02	Possible duplicate
Video 01	File integrity checks passed

Illustrative screening results, not actual verification.SECTION 06 — Validation & Review

Missing information, inconsistent answers, evidence warnings and unresolved fields.

SECTION 07 — Supporting Documents

List of submitted files, evidence IDs, file hashes and attachment references.

SECTION 08 — Claim Preparation Summary

Clear description of the incident, available evidence, outstanding requirements and next steps.

SECTION 09 — Declaration & Confirmation

Customer confirmation, report generation timestamp, report ID and version.

Report ready for customer download

ClaimSathi AI preparation report. Not an insurer-issued claim form, surveyor report, claim approval or guarantee of settlement.

4. One critical architecture decision: Don't hardcode one insurer's format
You mentioned that the data should be stored in the format expected by actual insurance companies.

Correct objective. But different insurers and products can request different information and supporting documents.

We should implement two layers:

Layer 1 — ClaimSathi Canonical Schema

Our own standard, structured data model.

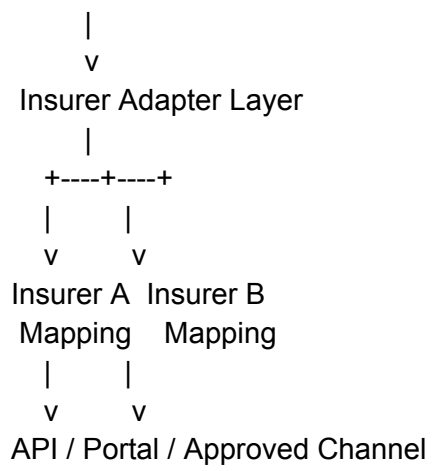
For example:

```
{
  "claim_id": "CS-2026-000124",
  "customer_id": "USR-100001",
  "insurance_type": "motor",
  "incident": {
```

```
    "date": "2026-11-05",
    "location": "Davanagere",
    "description": "Customer-reported vehicle accident"
  },
  "evidence": {
    "images": [],
    "videos": [],
    "integrity_checks": []
  },
  "validation": {
    "missing_fields": [],
    "inconsistencies": []
  },
  "customer_confirmation": true,
  "report_status": "generated",
  "insurer_submission_status": "not_submitted"
}
```


Layer 2 — Insurer-Specific Adapters

Later, when we sign a partnership:



This means we don't need to rebuild our entire application whenever we onboard another insurer.

Our standard dossier can work today. Partner-specific field mappings and official submission formats can be added later.

4. Login, unique IDs and lifetime customer records

This is another core part of the product, not an optional dashboard feature.

Each customer should have a persistent account and private claims vault.
Customer Claims Vault
Rahul's account
Authenticated

Illustrative data
Customer IDUSR-100001

Total cases03

Previous claim records
CS-2026-000124
Report ready

Motor accident · November 5, 2026Report · 4 images · 1 video

CS-2026-000085
Archived

Previous claim · September 2026
Available customer actions
Download previous reports

View uploaded evidence

View claim history and report versions

Suggested database structure

Table	Responsibility
users	Customer accounts and unique IDs
user_consents	Consent and privacy preferences
conversations	Conversation sessions
claims	Structured claim information
claim_fields	Extracted and confirmed fields
evidence_assets	Image/video metadata and storage references
evidence_checks	AI provenance, hashes and duplicate results
reports	Generated PDF and JSON reports
claim_status_events	Historical status changes

audit_logs Access and modification history

Storage recommendation: Keep structured information in the database and actual images/videos in encrypted object storage. Do not store large media files directly as database blobs.

For long-term customer access, implement encryption, access controls, backups, consent records, export and deletion mechanisms, and a clearly defined retention policy. We can design a long-lived claims vault, but shouldn't promise indefinite retention without accounting for legal and operational retention requirements.
5. How will insurance companies actually receive and process our reports?
This is the future business integration, and we should explain it properly in the MVP submission.

First, understand that ClaimSathi AI is not an insurance company. We are a technology platform preparing and screening claim information.

The actual insurer remains responsible for official claim registration, policy verification, assessment and settlement.
Future integration architecture
ClaimSathi AI
Confirmed claim dossier · PDF · Structured JSON · Evidence references

Insurer Integration Gateway
Customer-authorized routing · Insurer-specific data mapping · Authentication

Insurance Company A
Partner API

Insurance Company B
Partner API

Insurance Company C
Secure partner portal

Other Insurers
Approved integrations

Insurer's Claims Management System
Official case registration and processing

How communication would work
I recommend this priority:

Method	How it works	Priority
Official insurer API	Submit claim JSON and evidence through authenticated endpoints	Preferred
Secure partner portal	Authorized submission through insurer portal	Alternative
Secure SFTP	Encrypted file exchange under a partner agreement	Alternative
Email	Only through an expressly approved secure process	Last resort

We should not automatically send customers' personal information to random insurer email addresses.
How an insurer resolves the case
Consider the future workflow:
1. ClaimSathi sends the case through an authorized integration.
2. Insurer acknowledges receipt.
3. Insurer generates its own official claim reference number.
4. Insurer checks the policy and applicable coverage.
5. Insurer requests any additional information.
6. A surveyor or authorized assessor may inspect the loss, as applicable.
7. Insurer evaluates the claim under its policy and applicable procedures.
8. Insurer communicates the decision and settlement information to the customer.
IRDAI's regulatory framework requires insurers to operate under board-approved claim-settlement policies and provide transparent assessments and settlement processes.
Press Information Bureau

ClaimSathi can track these updates through insurer webhooks/API responses or authorized status updates. We must not claim a case has been received, approved or settled until there is an actual confirmation.
What do we demonstrate without partnerships?
This is a very important distinction for your final submission.

Build a Mock Insurer Integration Gateway.

It should simulate the future communication:
ClaimSathi Report Generated
|
v
Mock Insurer API
|
v
Payload Received
|
v
Mock Insurer Acknowledgement
|

v
Demo Claim Reference Generated
|
v
Mock Processing Status

For example:

```
{
  "submission_id": "DEMO-SUB-1001",
  "claim_id": "CS-2026-000124",
  "recipient": "DEMO_INSURER_A",
  "delivery_status": "simulated_success",
  "insurer_claim_reference": "DEMO-INS-001",
  "processing_status": "received"
}
```

Demo only — not a real insurer submission
This lets you demonstrate the architecture convincingly without falsely claiming a partnership or real insurance-company integration.

In the final submission, explain that production onboarding requires insurer agreements, API specifications, data-sharing permissions, security reviews, testing and partner approval.

6. What exactly should we implement for the current MVP?
Here is the development scope I would lock.
ClaimSathi AI — MVP Scope
Must implement
Phase 1 — Customer platform
- Customer registration/login.
- Unique customer ID.
- Personal dashboard.
- Previous claim history.
- New claim initiation.

Phase 2 — Intelligent conversation
- Gnani STT and TTS.
- Voice and text interaction.
- Multilingual conversation.
- Structured claim extraction.
- Missing-field detection.
- Dynamic follow-up questions.

Phase 3 — Evidence collection
- Image upload.
- Video upload.
- Live camera capture.
- Evidence inventory.
- Secure original-file storage.

Phase 4 — Evidence integrity
- SHA-256 duplicate detection.
- Perceptual image duplicate detection.
- C2PA provenance verification.
- Supported SynthID verification.
- Metadata inspection.
- Suspicious-evidence flagging.
- Human-review recommendations.

Phase 5 — Report generation
- Professional report generation.
- Structured JSON export.
- PDF generation.
- Evidence references and verification results.
- Customer confirmation.
- Report version management.

Phase 6 — Record management
- Customer claims vault.
- Downloadable historical reports.
- Evidence history.
- Claim status tracking.
- Secure data access.

MVP demonstration feature
Phase 7 — Simulated insurer integration
- Mock insurer API.
- Submission payload mapping.
- Mock acknowledgement.
- Simulated insurer claim reference.
- Processing-status demonstration.

7. Recommended technology architecture
Considering our proposed React, FastAPI and Gnani stack:

Component	Suggested implementation
Frontend	React + TypeScript + Vite
Authentication	JWT + secure password/OTP authentication

Backend	FastAPI
Database	SQLite for local MVP; PostgreSQL for production
Voice	Gnani Prisma STT + Timbre TTS
AI conversation	LLM orchestration + controlled conversation state
Data extraction	Pydantic structured schemas
Image processing	Pillow + OpenCV
Exact duplicate detection	SHA-256
Visual duplicate detection	pHash
Image provenance	C2PA + supported SynthID checks
Media storage	Encrypted S3-compatible object storage
Report generation	HTML templates + PDF renderer
Future insurer integration	FastAPI adapter layer + authenticated APIs

8. What we should NOT build for this MVP

Let's keep the scope controlled.

Feature	Decision
Deepfake detection	Not required
Face recognition	Not required
Advanced video authenticity detection	Not required
Automated fraud conviction	Not required
Automated insurance approval	Not required
Settlement processing	Not required
WhatsApp bot	Future phase
Real insurance company integrations	Future partnerships

One recommendation: start with motor insurance claims for the first complete workflow. It gives us a concrete use case for vehicle photographs, damage videos, evidence reuse detection and report generation. We can expand into health, travel and other insurance categories through separate schemas later.

9. How do we define a successful MVP?

I would establish these acceptance criteria before declaring development complete.

Test scenario	Expected result
Customer explains an accident	AI extracts structured information
Important information is missing	AI asks appropriate questions
Customer uploads a valid photograph	File is stored and inspected
Customer uploads an exact duplicate	System flags the duplicate
Customer uploads a resized duplicate	pHash detects a possible match
Supported AI provenance is found	System records the signal
No AI provenance is found	System reports inconclusive/no supported signal
Customer captures live evidence	Capture session is recorded
Customer confirms the claim	Claim record is finalized
Report is generated	PDF and JSON are available
Customer logs in again	Previous records are accessible
Demo insurer receives submission	Simulated acknowledgement appears

We should also test false positives and false negatives against a labelled collection of original, AI-generated, resized, compressed and reused images. Don't set an accuracy percentage until those tests are actually completed.

10. The final product positioning
I would use this as the central product description for our development documentation and MVP submission:
ClaimSathi AI
Intelligent Insurance Claims Assistance & Evidence Integrity Platform
ClaimSathi AI is a multilingual, voice-first insurance claims platform that helps customers report incidents through natural conversation, automatically extracts and validates claim information, collects supporting photographs and videos, performs preliminary evidence-integrity screening, and generates professionally structured insurance claim reports.

The platform maintains a secure customer claims vault with persistent records, downloadable reports and evidence histories.

Its architecture is designed to support future integration with multiple insurance companies through insurer-specific APIs and authorized communication channels, enabling secure claim submission and status tracking.

My final architectural decision
I would separate this product into three core engines:
Claim Intelligence Engine
Conversation, extraction, validation and follow-ups.

Evidence Integrity Engine
Provenance, duplicates, file integrity and review flags.

Claim Documentation Engine
Professional reports, structured records and customer vault.

The insurer integration layer sits beyond these three engines, ready for future partnerships.

One final distinction: ClaimSathi AI prepares a claim dossier; the insurance company registers and adjudicates the official claim. That's the boundary we must preserve.

I would now freeze this as our MVP product specification and start implementation against it, rather than adding unrelated features.